

Dota 2 Trajectory Analyzer

Compression MDL de trajectoires spatio-temporelles

Elias OKAT Uzay TURKMENEL Paul AUBERT

Université de Caen Normandie — L3 Informatique

Projet 2 — 2025-2026

Contexte : DotA 2 et les données

DotA 2 — MOBA compétitif, 2×5 joueurs

Chaque match $\approx$ 30-45 min $\Rightarrow$ **flux massif** de (X, Y)

Trois obstacles majeurs :

- ❶ **Volume** — $\sim 10\text{M}$ pts / corpus
- ❷ **Bruit** — micro-déplacements
- ❸ **Pas de sémantique** — un point ne dit rien



Carte officielle de DotA 2

Question

Clics $\rightarrow$ **comportements stratégiques**
non supervisés ?

Pourquoi les approches existantes ne suffisent pas

1. Heatmaps statiques

- Montrent où les joueurs vont. . .
- Écrasent le temps : pas de séquence
- Jungle → Mid → Rivière ? Invisible.

2. Grilles manuelles

- Découpage arbitraire de la carte
- Biais humain aux frontières
- Ne s'adapte pas aux flux réels

3. Clustering direct sur points bruts

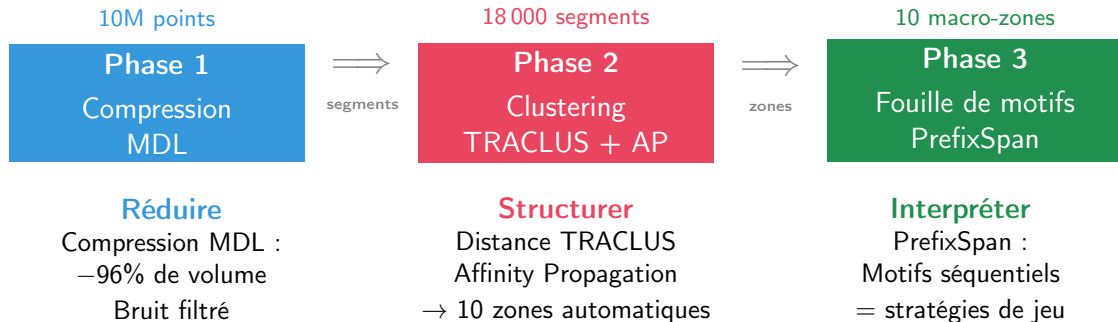
- Matrice $N \times N$: $N = 10M \Rightarrow$
impossible
- Le bruit noie le signal

Constat

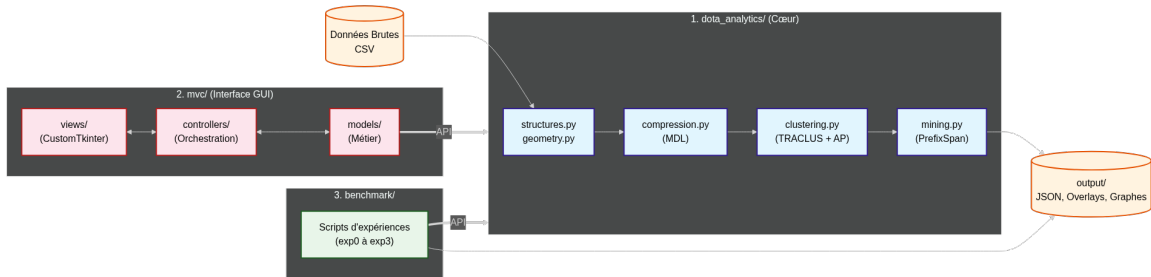
Il faut d'abord **compresser** les données pour rendre toute analyse possible.

⇒ Notre approche : un pipeline en 3 phases

Notre pipeline en 3 phases



Architecture logicielle — Diagramme de classes



Pourquoi Python ?

- Écosystème **data science** mature
- Prototypage rapide, code lisible
- Interopérabilité C/Fortran via NumPy

NumPy — calcul matriciel

- Matrices de distance $N \times N$ (TRACCLUS)
- Vectorisation : $\times 100$ vs boucles Python
- Mémoire efficace (float32)

Matplotlib — visualisation

- Superposition trajectoires sur carte
- Graphiques multi-panneaux (benchmarks)

Autres dépendances :

- scikit-learn — Silhouette, Davies-Bouldin
- scipy — Hausdorff, distances spatiales

Bilan

Zéro dépendance lourde !

Idée clé (Minimum Description Length)

La description la plus courte qui reste fidèle à la trajectoire (erreur $< w_{error}$).

Algorithme glouton :

- 1 Partir du premier point
- 2 **Étendre** le segment point par point
- 3 Calculer $d_{\perp}$ de chaque intermédiaire
- 4 $\max(d_{\perp}) < w_{error} \Rightarrow$ continuer ✓
- 5 $\max(d_{\perp}) \geq w_{error} \Rightarrow$ **couper** ✗
- 6 Reprendre au dernier point validé

w_{error} = tolérance max :

- Petit $\rightarrow$ haute fidélité, peu de compression
- Grand $\rightarrow$ forte compression, perte de détail
- **Notre choix** : $w_{error} = 12$

Complexité : $\mathcal{O}(N^2)$ pire cas, $\sim \mathcal{O}(N^{1.5})$ en pratique

Résultat

Compression **96–98%** du volume, bruit filtré, erreur bornée.

Résultat : avant / après compression



Trajectoire brute

Milliers de points, bruit inclus



Après MDL ($w_{error} = 12$)

~250 segments / joueur

Comment **prouver** que le pipeline produit des résultats fiables ?

Exp. 1 — Compression MDL

- Corpus : **70 matchs** réels
- Vérif. erreur max $< w_{error}$ ✓
- Robustesse au bruit gaussien
- Comparaison : *Uniforme*,
Douglas-Peucker
- Sensibilité de w_{error}

Exp. 2 — Clustering

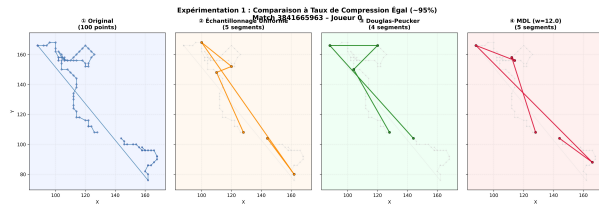
- Recherche du k optimal (coude)
- K-Means vs K-Médoïdes vs AP
- Sensibilité à N (AP)

Exp. 3 — Fouille PrefixSpan

- Extraction de motifs séquentiels
- Interprétation stratégique sur carte

⇒ Chaque choix de paramètre est **justifié expérimentalement** (pas de valeur arbitraire).

Exp. 1 — MDL vs Uniforme vs Douglas-Peucker



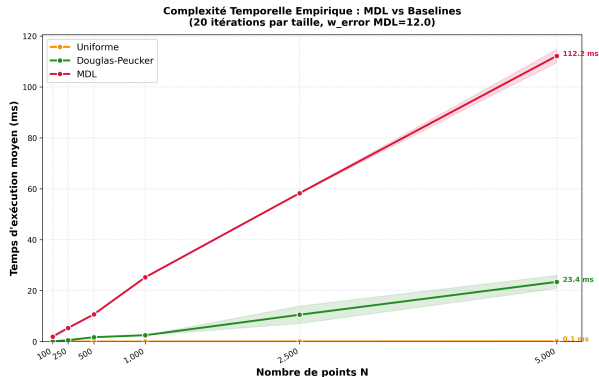
À compression égale (~95%) :

Méthode	RMSE	Hausd.
Uniforme	5.07	12.80
Douglas-P.	10.81	22.09
MDL	5.48	11.66

Verdict

MDL : **meilleure fidélité** (Hausdorff min).
Douglas-Peucker piégé par les outliers.

Exp. 1b — Complexité temporelle : MDL vs Baselines



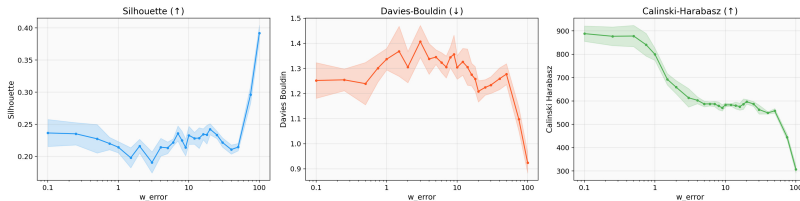
Méthode	Complexité	5 000 pts
Uniforme	$\mathcal{O}(1)$	0.1 ms
Douglas-P.	$\mathcal{O}(N \log N)$	23 ms
MDL	$\mathcal{O}(N^{1.5})$	112 ms

Verdict

MDL plus lent mais **acceptable** ($\sim 7s$ / match). Le coût se justifie par la **qualité**.

Exp. 2 — Sensibilité de w_{error} : pourquoi 12 ?

Fig 0.4 — Métriques de clustering vs w_{error}



Observation :

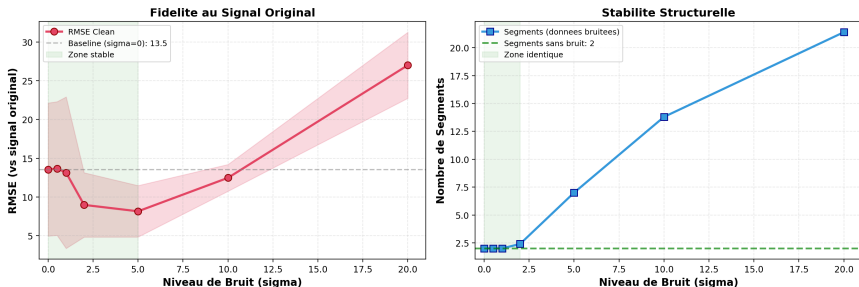
- w_{error} petit → fidélité mais faible compression
- w_{error} grand → plateau à $\sim 96\%$
- **Coude** à $w_{error} \approx 12$

Conclusion

$w_{error} = 12$ = meilleur compromis
compression / fidélité.
Silhouette se stabilise, bruit $\sigma \leq 2$ absorbé.

Exp. 3 — Robustesse de MDL au bruit gaussien

Robustesse MDL au Bruit Gaussien ($w_{\text{error}} = 12$)

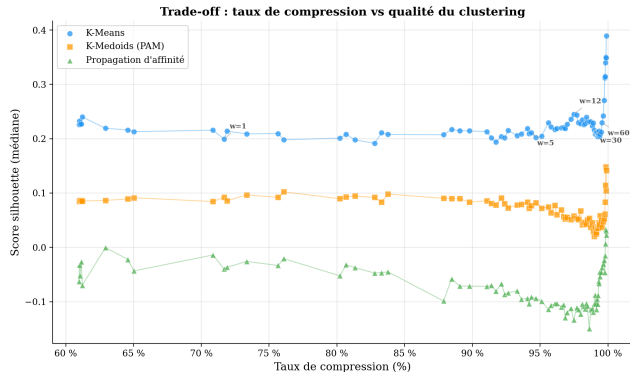


Protocole : injection de bruit $\mathcal{N}(0, \sigma^2)$ croissant sur 5 échantillons, compression MDL ($w_{\text{error}} = 12$).

Résultat

RMSE stable jusqu'à $\sigma = 5$.
Structure identique (~ 2 seg.) jusqu'à $\sigma = 2$.
MDL **filtre le bruit**, pas le signal.

Compression → Clustering : quel impact ?



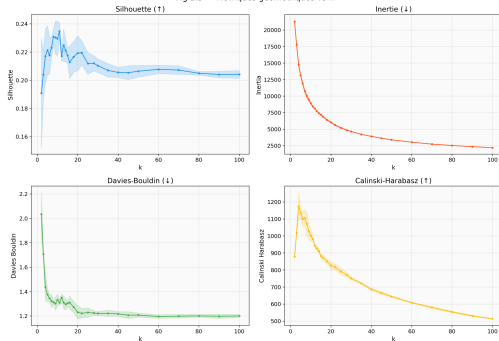
Chaque point = un w_{error} différent.
 $w_{error} = 12$ annoté à $\sim 96\%$ de compression.

Conclusion

La compression MDL **ne dégrade pas** le clustering. $w_{error} = 12$ est dans la zone stable.

Optimisation de k : compromis topologie / sémantique

Fig 1.3 — Métriques géométriques vs k



Recherche du k optimal ($k \in [2, 30]$)

Méthode du coude :

- **Davies-Bouldin** : minimum local à $k \approx 10$ ✓
- **Silhouette** : stabilisation en plateau

Validation par PrefixSpan

À $k = 10$, volume de motifs **riche mais maîtrisé** : pas d'explosion combinatoire.

⇒ **Choix final** : $k = 10$ zones

Évaluation comparative des algorithmes de clustering

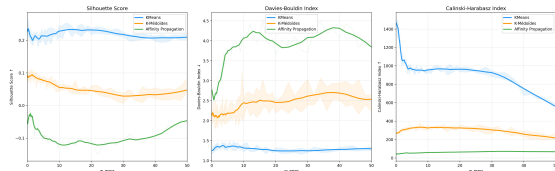
Trois algorithmes en compétition :

- 1 K-Means (baseline euclidienne)
- 2 K-Médoïdes (PAM)
- 3 Affinity Propagation (AP)

Avantage de la distance TRACLUS :

- K-Means limité à l'**euclidien brut**
- K-Médoïdes / AP exploitent la **matrice experte** (perp. + par. + angle)

Comparison KMeans / K-Médoïdes / Affinity Propagation



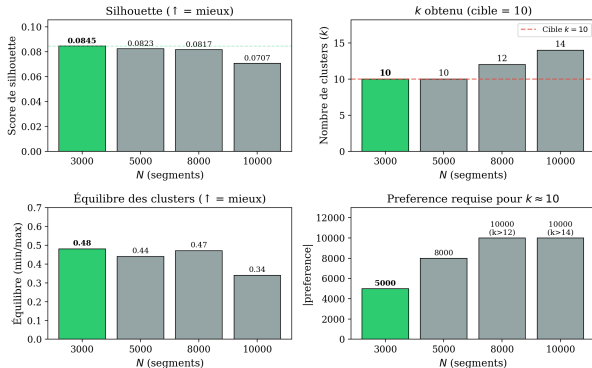
Scores de Silhouette par algorithme

Observation

K-Médoïdes et AP **doublent** le score Silhouette de K-Means grâce à TRACLUS.

Sensibilité de l'AP à la taille de l'échantillon

Sensibilité d'Affinity Propagation au nombre de segments (N)



Vert = configuration optimale retenue ($N=3000$, preference=-5000, $k=10$). Pour $N \geq 8000$, AP ne peut plus atteindre $k=10$.

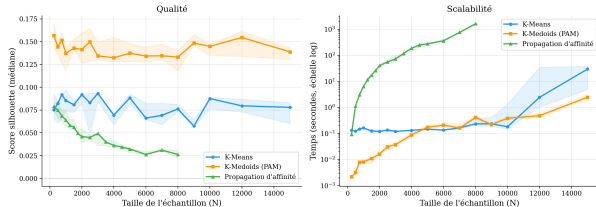
- **Passage à l'échelle** : sur-segmentation (12–14 clusters) dès $N \geq 8000$
- **Zone de convergence** : $k = 10$ stable pour $N = 3000$

Décision technique

Sous-échantillonnage à $N = 3000$ segments pour un découpage cohérent en 10 zones.

Benchmark : qualité vs scalabilité ($k = 12$)

Comparaison des 3 algorithmes de clustering ($k = 12$)



Qualité (Silhouette)

- **K-Médoids** domine (médiane ≈ 0.15)
- **AP** : instable si $N > 8\,000$

Scalabilité (temps)

- **K-Means / K-Médoids** : quasi linéaires
- **AP** : $\mathcal{O}(N^2)$, prohibitif $> 4\,000$ seg.

Benchmark à $k = 12$ fixé. En production, l'AP converge vers $k = 10$.

Le paradoxe : pourquoi rejeter le K-Médoïdes ?



L'AP identifie 10 zones distinctes, sans doublons.

Le problème des doublons :

- Deux centres à < 5 unités d'écart
- **Gaspillage** : même zone couverte deux fois (Offlane Radiant)
- Zones entières ignorées (Top Lane)

Limite du Silhouette

Un bon score statistique $\neq$ **cohérence géographique**.
K-Médoïdes optimise la compacité,
pas la couverture.

Choix final : Affinity Propagation paramétré



Les 10 macro-zones découvertes par l'AP

Pourquoi l'AP malgré son coût ?

Seul algorithme produisant un découpage **géographiquement cohérent**, servant d'**alphabet** pour PrefixSpan.

Paramétrage retenu :

- $N = 3\,000$ segments (convergence)
- **Damping** = 0.7 (stabilisation)
- **Préférence** = $-5\,000$ (cible $k \approx 10$)

⇒ **Découpage 100% guidé par la donnée**

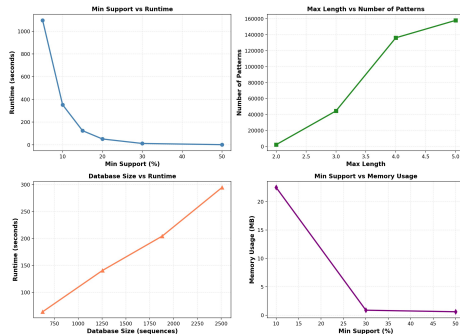
Principe :

- Trouver les sous-séquences d'états fréquentes
- Approche par *pattern growth*

Paramètres :

- minSupport : $0.2 = 20\%$
- maxLength : 5

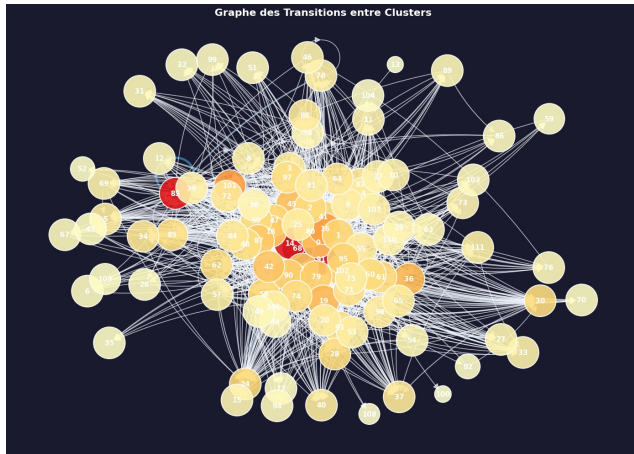
Benchmark PrefixSpan



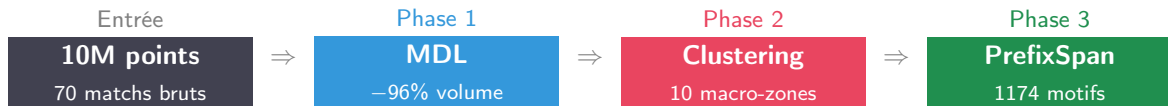
Résultats — Motifs et graphe de transitions

Motif	Sup.
14 → 68	22
91 → 68	24
91 → 36	21
68 → 83	19

Interprétation : Zones 14, 68, 91 = hubs stratégiques



Bilan : de 10M de clics aux stratégies



Résultats clés :

- Compression **96–98%** fidèle (erreur < 12 unités)
- Clustering AP : **10 zones** sans doublons ni trous
- Motifs séquentiels interprétables : cycles de runes, rotations jungle, corridors de transition

Choix méthodologiques validés :

- Chaque paramètre justifié par benchmark
- AP et PrefixSpan codés *from scratch*
- 72 tests unitaires, graine fixée (SEED=42)
- Pipeline reproductible bout-en-bout

Ce que l'on peut en tirer

Enseignements techniques

- Un **bon score statistique** (Silhouette) ne garantit pas la **cohérence géographique**
- La compression est un **prérequis**, pas une option : sans MDL, aucun clustering n'est viable à cette échelle
- Le sous-échantillonnage maîtrisé ($N=3\,000$) suffit pour capturer la structure globale

Enseignements métier (DotA 2)

- Les **rotations de runes** et les **corridors mid→jungle** dominent les motifs fréquents
- Les stratégies émergent de la donnée **sans supervision** ni connaissance experte
- Le pipeline est **transposable** à d'autres jeux ou données spatio-temporelles

Conclusion générale

De la donnée brute (18 000 segments) aux motifs interprétables (1 174 séquences), le pipeline a su **préserver et révéler une structure latente** que la seule observation visuelle ne permettait pas de saisir.

Enrichissement des données

- **Étiquetage Radiant / Dire** : séquences synchrones par équipe pour extraire des stratégies multi-agents
- **Filtrage par rôle** (carry, support, mid...) : comparer les schémas de déplacement par poste
- **Événements de jeu** (kills, achats, tours) : relier les déplacements aux décisions stratégiques

Améliorations algorithmiques

- **DBSCAN / HDBSCAN** : clustering sans fixer k , formes non sphériques
- **Fenêtre temporelle** : détecter les phases de jeu (early / mid / late game)
- **Compression adaptative** : w_{error} variable selon la densité locale

Vision

Un outil d'analyse temps réel pour les équipes professionnelles et les casters.

Merci de votre attention

Elias OKAT · Uzay TURKMENEL · Paul AUBERT

Université de Caen Normandie — L3 Informatique — Projet 2

Questions ?